

UNIVERSIDAD EAFIT

ESCUELA DE INGENIERÍA
INGENIERÍA DE SISTEMAS

ESTRUCTURAS DE DATOS Y ALGORITMOS I

PRÁCTICA 1

Estructuras Unidimensionales

Pilas y colas aplicadas a un sistema de Undo/Redo
y al control de tráfico de un firewall

Integrantes

Simon Ospina Ruiz

Samuel Lopez Velez

Cristian Gustavo Buitrago Silva

Docente: Carlos Alberto Alvarez Henao

Medellín, Colombia

31 de agosto de 2026

Contenido

El contenido está organizado por temas, empezando por el contexto de los problemas y terminando con las conclusiones, referencias y contribuciones de cada integrante.

1. Introducción
2. Contexto de los problemas
 - 2.1 Sistema de Undo/Redo
 - 2.2 Búfer y limitador de tasa
3. Fundamentación teórica
 - 3.1 Estructuras unidimensionales
 - 3.2 Pilas
 - 3.3 Colas
 - 3.4 Listas enlazadas
 - 3.5 Arreglos y memoria dinámica
 - 3.6 Tipo Abstracto de Datos (TAD)
4. Diseño de las soluciones
 - 4.1 Problema 1: Undo/Redo
 - 4.2 Problema 2: Firewall
 - 4.3 Decisiones de diseño
5. Pseudocódigo
6. Análisis de complejidad
 - 6.1 Complejidad de la pila
 - 6.2 Complejidad de las colas
 - 6.3 Complejidad del RateLimiter
 - 6.4 Complejidad global
 - 6.5 Análisis amortizado
7. Casos de prueba
8. Experimentación y limitaciones
9. Comparación teórico-experimental
10. Conclusiones
11. Uso de herramientas de IA
12. Referencias
13. Contribución individual

1. Introducción

Las estructuras de datos nos ayudan a organizar la información de una forma que facilite trabajar con ella. En esta práctica trabajamos principalmente con pilas y colas y las relacionamos con dos situaciones que pueden aparecer en programas reales: el sistema de Undo/Redo de un editor de código y el control de paquetes de un firewall.

En el primer problema se usa una pila porque necesitamos que la última modificación sea la primera en deshacerse. Esto corresponde al comportamiento LIFO (Last In, First Out). En el segundo problema usamos colas porque los paquetes deben mantenerse en el orden en que llegan, es decir, FIFO (First In, First Out). También se utiliza una cola de timestamps para controlar los paquetes que siguen dentro de una ventana de tiempo.

Además de hacer que los programas funcionen, la práctica busca entender por qué funcionan así y cuánto cuesta ejecutar sus operaciones. Por eso se revisan las operaciones principales, los casos límite y la complejidad en tiempo y memoria. También se mira el caso de operaciones que algunas veces pueden costar más, como cuando una estructura necesita aumentar su tamaño.

2. Contexto de los problemas

2.1 Sistema de Undo/Redo

Un editor de texto o código debe permitir que el usuario pueda devolver el documento a un estado anterior. En nuestro caso se trabajan tres tipos de edición: INSERT, DELETE y REPLACE. Cada operación guarda la información necesaria para poder deshacerla y volverla a hacer.

Se manejan dos pilas: una para Undo y otra para Redo. Cuando se hace una nueva edición, esta pasa a Undo y Redo se limpia. Cuando se hace UNDO, se toma la última operación, se aplica su inversa y se pasa a Redo. Cuando se hace REDO, la operación vuelve al documento y regresa a Undo.

Esto funciona de esta manera porque una pila sigue el modelo LIFO: la última operación que entró es la primera que sale.

2.2 Búfer y limitador de tasa de un firewall

El segundo problema representa un firewall que recibe paquetes. Cada paquete tiene un timestamp y un tamaño en bytes. El búfer tiene una capacidad fija C y se implementa con una cola circular. Si el búfer ya está lleno, el siguiente paquete se rechaza.

También se lleva una cola con los timestamps de los paquetes recientes. Antes de revisar un nuevo paquete, se eliminan los timestamps que ya quedaron por fuera de la ventana de tiempo T .

Después se revisa si todavía hay menos de L timestamps. En el código, cuando $\text{timestamp} - \text{oldest}$ es mayor o igual que T , ese timestamp se considera expirado y se elimina.

La solución trabaja con un solo flujo de tráfico. No se agregó una búsqueda por dirección IP porque eso no hace parte del alcance principal de esta práctica.

3. Fundamentación teórica

3.1 Estructuras unidimensionales

Una estructura unidimensional guarda los elementos siguiendo un orden lineal. En esta práctica trabajamos con arreglos, listas enlazadas, pilas y colas. La diferencia principal está en la forma en que se pueden agregar, quitar y consultar los elementos.

3.2 Pilas

Una pila funciona como una pila de objetos: el último elemento que se agrega es el primero que se retira. Esto se conoce como LIFO. Sus operaciones principales son push para agregar, pop para sacar, top o peek para consultar el último elemento y isEmpty para saber si está vacía.

En el Problema 1 la pila se maneja con un arreglo dinámico. Empieza con una capacidad de 4 elementos y, cuando se llena, se crea un arreglo nuevo con el doble de espacio, se copian los elementos y se elimina el arreglo anterior. Aunque ese cambio toma más tiempo en algunas ocasiones, en general push mantiene un costo constante amortizado.

3.3 Colas

Una cola funciona como una fila: el primero que llega es el primero que sale. Esto se conoce como FIFO. Sus operaciones principales son enqueue para agregar, dequeue para sacar, front y rear para consultar los extremos, además de isEmpty e isFull cuando la cola tiene una capacidad fija.

En la cola circular, head marca el frente y tail indica dónde se puede insertar el siguiente elemento. Al llegar al final del arreglo, se vuelve a la primera posición, por eso se llama circular. En nuestro código también se usa count para saber cuántos elementos hay. Si count es 0, está vacía; si count es igual a capacity, está llena.

3.4 Listas enlazadas

Una lista enlazada está formada por nodos. Cada nodo guarda un dato y una referencia al siguiente nodo. En el proyecto se tienen head y tail, lo que permite saber rápidamente cuál es el primero y cuál es el último.

En la clase listas, enqueue crea un nuevo nodo y lo conecta al final. Dequeue toma el nodo del frente, mueve head al siguiente y libera el nodo anterior. Si ya no quedan elementos, tail también vuelve a nullptr.

3.5 Arreglos y memoria dinámica

Un arreglo guarda sus elementos en posiciones consecutivas de memoria y permite acceder directamente a una posición. En la pila dinámica se usa new[] para reservar memoria y delete[] para liberarla. Cuando se llena, se necesita crear un espacio más grande y copiar los elementos.

3.6 Tipo Abstracto de Datos (TAD)

Un Tipo Abstracto de Datos, o TAD, describe qué puede hacer una estructura sin importar cómo está construida por dentro. Por ejemplo, una pila tiene operaciones como push, pop, top e isEmpty.

La misma idea se puede implementar con un arreglo o con una lista. Para quien usa la estructura, lo importante es que las operaciones funcionen de la misma manera.

4. Diseño de las soluciones

4.1 Problema 1: Undo/Redo

La implementación define una estructura Operacion con los campos tipo, posicion, contenidoAnterior y contenidoNuevo. El documento se representa mediante un string. Para INSERT se almacena el contenido nuevo y el anterior queda vacío; para DELETE se conserva el contenido eliminado; y para REPLACE se separan el contenido anterior y el nuevo mediante el carácter '|'.

Las operaciones de edición se almacenan en Undo. Después de cualquier EDIT se vacía Redo. En UNDO se obtiene la operación superior, se aplica la transformación inversa y se mueve la operación a Redo. En REDO se aplica nuevamente la transformación original y se mueve la operación a Undo. Si la pila correspondiente está vacía, se informa un no-op y el programa continúa.

4.2 Problema 2: Firewall

El firewall trabaja con tres parámetros: C es la capacidad máxima del búfer, T es el tiempo de la ventana y L es la cantidad máxima de timestamps permitidos dentro de esa ventana.

La cola circular guarda temporalmente los tamaños de los paquetes aceptados. Para los timestamps se usa la cola enlazada de la clase listas. Antes de revisar cada paquete, RateLimiter llama a purgeExpired para quitar los timestamps que ya vencieron. Después, si quedan menos de L, el timestamp nuevo se agrega y el paquete puede pasar.

4.3 Decisiones de diseño

Decisión	Implementación	Justificación
Pila	Arreglo dinámico	Permite administrar capacidad manualmente y analizar el costo de redimensionamiento.
Undo/Redo	Dos pilas conceptuales	Permite representar de forma natural el historial de deshacer y rehacer.
Búfer	Cola circular	Mantiene FIFO y reutiliza posiciones del arreglo.
Timestamps	Lista enlazada usada como cola	Permite insertar al final y retirar del frente en $O(1)$.
Ventana temporal	Purga desde el frente	Los timestamps están ordenados por llegada; el más antiguo siempre está al frente.

Tabla 1. Principales decisiones de diseño.

5. Pseudocódigo

Los siguientes pasos muestran la idea de cada operación sin depender de la sintaxis de C++. Esto permite entender primero la lógica y luego relacionarla con el código.

5.1 Push en pila dinámica

```
PUSH(op)
1. Si cantidad == capacidad:
```

2. `capacidad <- capacidad * 2`
3. crear un nuevo arreglo con la nueva capacidad
4. copiar los elementos existentes al nuevo arreglo
5. liberar el arreglo anterior
6. `datos[cantidad] <- op`
7. `cantidad <- cantidad + 1`

5.2 Pop y Top

POP()

1. Si la pila no está vacía:
2. `cantidad <- cantidad - 1`
3. Si está vacía, no retirar ningún elemento.

TOP()

1. Si la pila no está vacía:
2. devolver `datos[cantidad - 1]`

5.3 Enqueue en cola circular

ENQUEUE(valor)

1. Si `count == capacity`:
2. devolver FALSO
3. `data[tail] <- valor`
4. `tail <- (tail + 1) MOD capacity`
5. `count <- count + 1`
6. devolver VERDADERO

5.4 Dequeue en cola circular

DEQUEUE()

1. Si `count == 0`:
2. devolver FALSO
3. `valor <- data[head]`
4. `head <- (head + 1) MOD capacity`
5. `count <- count - 1`
6. devolver valor

5.5 Purga de timestamps

PURGAR_EXPIRADOS(timestamp)

1. Mientras la cola de timestamps no esté vacía:
2. `oldest <- frente de la cola`
3. Si `timestamp - oldest >= T`:
4. retirar `oldest`
5. En caso contrario:
6. detener la purga

5.6 Allow del limitador

```
ALLOW(timestamp)
1. PURGAR_EXPIRADOS(timestamp)
2. Si tamaño de timestamps >= L:
3.     devolver FALSO
4. Insertar timestamp en la cola
5. devolver VERDADERO
```

5.7 Undo y Redo

```
UNDO()
1. Si Undo está vacía:
2.     reportar no-op
3. Si no:
4.     op <- TOP(Undo)
5.     aplicar la operación inversa sobre el documento
6.     POP(Undo)
7.     PUSH(Redo, op)
```

```
REDO()
1. Si Redo está vacía:
2.     reportar no-op
3. Si no:
4.     op <- TOP(Redo)
5.     reaplicar la operación sobre el documento
6.     POP(Redo)
7.     PUSH(Undo, op)
```

6. Análisis de complejidad

Para el análisis de complejidad se separa el trabajo propio de las estructuras del trabajo adicional que hacen el documento y el RateLimiter. También se tiene en cuenta que el arreglo de la pila puede necesitar redimensionarse y que la cola de timestamps puede eliminar varios elementos en una sola purga.

6.1 Complejidad de la pila

Operación	Mejor	Promedio / amortizado	Peor	Espacial	Justificación
push sin redimensionar	$\Omega(1)$	$\Theta(1)$	$O(1)$	$O(n)$	Escritura en la siguiente posición.
push con redimensionamiento	$\Omega(1)$	$\Theta(1)$ amortizado	$O(n)$	$O(n)$	Se crea un arreglo nuevo y se copian n elementos.
pop	$\Omega(1)$	$\Theta(1)$	$O(1)$	$O(n)$	Solo disminuye el contador.
top	$\Omega(1)$	$\Theta(1)$	$O(1)$	$O(n)$	Acceso directo al último elemento.
isEmpty	$\Omega(1)$	$\Theta(1)$	$O(1)$	$O(n)$	Comparación de cantidad con cero.

Tabla 2. Complejidad de las operaciones de la pila dinámica.

6.2 Complejidad de las colas

Operación	Cola circular	Cola enlazada	Justificación
enqueue	$\Theta(1)$	$\Theta(1)$	Se escribe en tail o se enlaza un nuevo nodo al final.
dequeue	$\Theta(1)$	$\Theta(1)$	Se avanza head o se retira el nodo del frente.
front	$\Theta(1)$	$\Theta(1)$	Acceso directo a head.
rear	$\Theta(1)$	$\Theta(1)$	Acceso directo a tail.
isEmpty	$\Theta(1)$	$\Theta(1)$	Consulta de count o head.
isFull	$\Theta(1)$	No aplica	Se compara count con capacity.

Tabla 3. Complejidad de las operaciones de las colas.

6.3 Complejidad del RateLimiter

purgeExpired puede sacar varios timestamps de una sola vez. Eso significa que una llamada puede tardar más si hay muchos elementos vencidos. Aun así, cada timestamp que entra a la cola solo se elimina una vez, por lo que al mirar todos los paquetes el trabajo total de las purgas crece de forma lineal. Por eso, el costo amortizado de la purga por paquete es $\Theta(1)$.

6.4 Complejidad global

Componente	Tiempo estructural	Espacio
Undo/Redo con pila dinámica	$\Theta(n)$ amortizado para las operaciones de pila	$O(n)$ en el peor caso
Búfer circular	$\Theta(n)$ para n operaciones	$O(C)$
Cola de timestamps + purga	$\Theta(n)$ amortizado	$O(L)$ como máximo bajo el límite de ventana
RateLimiter completo	$\Theta(n)$ amortizado	$O(L)$

Tabla 4. Complejidad estructural global.

En el Problema 1 hay algo adicional: el documento se guarda en un `std::string`. Dependiendo del cambio que se haga, también puede ser necesario mover o copiar caracteres. Por eso, el tiempo total de una edición no depende solamente de la pila, sino también del tamaño del documento.

6.5 Análisis amortizado

El crecimiento de la pila es un ejemplo de análisis amortizado. Algunas veces push tiene que crear un arreglo nuevo y copiar todos los elementos, por lo que esa operación tarda más. Después de aumentar la capacidad, se pueden hacer varias inserciones sin volver a copiar. Por eso, si se mira el conjunto de operaciones, push tiene un costo amortizado de $\Theta(1)$.

La purga del RateLimiter es otro caso amortizado. Una llamada puede retirar varios timestamps, pero cada uno de ellos se elimina una sola vez. Por eso, aunque una llamada individual pueda costar $O(k)$, el trabajo total de todas las purgas sigue siendo lineal.

7. Casos de prueba

Los casos de prueba sirven para comprobar que el programa funcione tanto en situaciones normales como en situaciones límite. Se revisan especialmente las estructuras vacías, los límites de capacidad, los excesos de tasa y el comportamiento de Undo/Redo.

7.1 Problema 1 — Undo/Redo

ID	Caso	Comportamiento esperado
P1-01	Secuencia mixta	EDIT, UNDO y REDO mantienen el documento coherente.
P1-02	UNDO con pila vacía	Se reporta no-op y el programa continúa.
P1-03	Una edición, UNDO y REDO	La modificación se revierte y posteriormente se recupera.
P1-04	EDIT después de UNDO	Redo se vacía completamente.
P1-05	Múltiples UNDO	Se deshacen las operaciones en orden inverso hasta vaciar Undo.
P1-06	Más REDO que disponibles	Los REDO posteriores se reportan como no-op.
P1-07	Crecimiento	La capacidad aumenta cuando se alcanza el límite.

Tabla 5. Casos de prueba del Problema 1.

7.2 Problema 2 — Firewall

ID	Caso	Comportamiento esperado
P2-01	Flujo normal	Los paquetes que cumplen capacidad y tasa son aceptados.
P2-02	Búfer vacío	isEmpty devuelve verdadero.
P2-03	Un paquete	El paquete se encola correctamente.
P2-04	Búfer lleno	Un paquete adicional no puede ser encolado.
P2-05	Ráfaga sobre L	Los paquetes que superan el límite se rechazan por tasa.
P2-06	Deque vacío	La operación devuelve falso sin provocar una caída.
P2-07	Borde temporal	Un timestamp con diferencia exactamente T se considera expirado en esta implementación.

Tabla 6. Casos de prueba del Problema 2.

8. Experimentación y limitaciones

La experimentación no se incluye con resultados numéricos en este informe porque no se tienen mediciones reales guardadas del proyecto. Por esta razón, se evita poner tiempos o gráficas que no hayan sido obtenidos por el equipo.

El proyecto sí cuenta con un generador de datos en `data/generador.cpp`. Este permite crear paquetes con una cantidad definida y usando una semilla, lo que ayuda a que los datos puedan repetirse.

Para medir tiempos con tamaños grandes, el programa tendría que ejecutar todos los paquetes de cada prueba. En la versión revisada, la ejecución principal limita el procesamiento a dos paquetes, por lo que esa parte tendría que cambiarse antes de hacer una medición completa.

8.1 Recomendación para completar la experimentación

Tamaño sugerido	Repeticiones mínimas	Mediciones
1.000	5	Tiempo total, métrica del dominio
10.000	5	Tiempo total, métrica del dominio
100.000	5	Tiempo total, métrica del dominio
1.000.000	5	Tiempo total, métrica del dominio

Tabla 7. Plan de experimentación recomendado por la guía.

9. Comparación teórico-experimental

Como no hay mediciones experimentales guardadas, la comparación se limita a lo que se espera según la teoría. Las operaciones básicas de pilas y colas deberían mantener un costo constante, mientras que el crecimiento de la pila y algunas purgas pueden generar operaciones puntualmente más costosas.

9.1 Arreglo frente a lista enlazada

Aunque algunas operaciones pueden ser $\Theta(1)$ tanto en un arreglo circular como en una lista enlazada, en la práctica el arreglo puede ser más rápido porque sus datos están juntos en memoria. En la lista enlazada hay que seguir referencias entre nodos que pueden estar en lugares diferentes.

9.2 Constantes ocultas y memoria

La notación O , Θ u Ω nos ayuda a ver cómo crece el costo cuando aumenta la cantidad de datos, pero no dice exactamente cuánto tardará un programa en una máquina. El compilador, las optimizaciones, la memoria y el procesador también pueden cambiar los tiempos.

9.3 Comportamiento amortizado

En la pila puede haber un aumento puntual del tiempo cuando se duplica la capacidad. En el RateLimiter también puede haber una purga que elimine varios timestamps. Esto no contradice el análisis amortizado, porque esos costos se distribuyen entre muchas operaciones.

10. Conclusiones

1. La práctica permitió ver cómo una estructura de datos puede adaptarse a un problema real. En Undo/Redo se usa una pila por su comportamiento LIFO y en el firewall se usan colas por su comportamiento FIFO.
2. La pila dinámica también permite entender la diferencia entre el costo de una operación puntual y el costo amortizado. Aunque algunas veces se deben copiar varios elementos, aumentar la capacidad ayuda a que push mantenga un costo amortizado constante.
3. En la cola circular se usa un contador para distinguir cuando está vacía y cuando está llena. Esto evita confundir los dos estados cuando head y tail coinciden.
4. El RateLimiter muestra otro caso de análisis amortizado. Una purga puede sacar varios timestamps de una vez, pero cada timestamp solo se elimina una vez, así que el trabajo total sigue siendo lineal.
5. Los casos límite fueron importantes para comprobar qué pasa cuando una estructura está vacía, cuando el búfer se llena o cuando un paquete llega justo en el límite de tiempo. Revisar estas situaciones ayuda a evitar errores en la ejecución.
6. Finalmente, la práctica mostró que no basta con que el código funcione. También es importante explicar cómo se tomó cada decisión, analizar su costo y probar los casos que pueden causar errores.

11. Uso de herramientas de IA

Durante el desarrollo de la práctica se utilizó una herramienta de Inteligencia Artificial como apoyo para resolver dudas y organizar algunas partes del trabajo. Su uso se concentró principalmente en el Problema 2.

11.1 Uso en el Problema 2

En el Problema 2 se utilizó IA para recibir ayuda con la estructura lógica, hacer preguntas puntuales sobre el código y resolver dudas que aparecieron durante la implementación. También se utilizó como apoyo para organizar y construir el main del segundo problema.

La IA se usó principalmente para orientar el proceso. Las respuestas sirvieron para aclarar la lógica y buscar posibles errores, pero el código fue revisado por el equipo para entender cómo funcionaba y por qué se tomaron las decisiones usadas.

11.2 Uso en la elaboración del informe

En la elaboración del informe también se utilizó IA para organizar las ideas y mejorar la claridad de algunas explicaciones. La revisión final del contenido quedó a cargo del equipo.

11.3 Preguntas orientadoras utilizadas durante el desarrollo

¿Por qué separar el proyecto en .hpp y .cpp?

Se separan para tener más organizado el código. En los .hpp se deja lo que la estructura ofrece y en los .cpp se escribe cómo funcionan esas operaciones. Esto hace más fácil leer el proyecto y encontrar dónde está cada parte.

¿Cómo sabes si la cola está vacía?

En la cola circular se revisa count. Si `count == 0`, no hay elementos y la cola está vacía. En la cola enlazada se revisa head: si `head == nullptr`, significa que no hay ningún nodo.

¿Por qué no basta con comparar head y tail para saber si está llena?

Porque head y tail pueden coincidir en dos situaciones diferentes: cuando no hay elementos y cuando la cola ya ocupó todas las posiciones. Por eso se usa count. Si `count == 0` está vacía y si `count == capacity` está llena.

¿Qué diferencia hay entre una cola circular y una cola enlazada?

Las dos siguen FIFO, pero se construyen de forma diferente. La cola circular usa un arreglo con una capacidad definida y reutiliza las posiciones. La enlazada usa nodos que se van creando dinámicamente, por lo que puede crecer mientras haya memoria disponible.

¿Qué hace `purgeExpired()`?

`purgeExpired(timestamp)` revisa desde el frente de la cola y va quitando los timestamps que ya vencieron. En el código, un timestamp se elimina cuando la diferencia con el actual es mayor o igual

que T. Así, antes de aceptar un paquete, solo quedan los timestamps que todavía pertenecen a la ventana.

¿Por qué se purgan primero los timestamps antiguos?

Porque los timestamps están guardados en el orden en que llegaron. El que está al frente siempre es el más viejo. Si ese ya venció, se puede eliminar y revisar el siguiente. Así se van quitando solamente los que ya no sirven para controlar la ventana de tiempo.

Para el informe se utilizó IA como apoyo para organizar las ideas y hacer más claras algunas explicaciones sobre pilas, colas y complejidad. El contenido fue revisado y adaptado para que corresponda con el trabajo realizado por el equipo.

12. Referencias

Universidad EAFIT. (2026). Estructuras de Datos y Algoritmos I — Práctica 1: Estructuras Unidimensionales. Documento de la asignatura.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.

Stroustrup, B. (2013). The C++ Programming Language (4th ed.). Addison-Wesley.

ISO. (2020). ISO/IEC 14882:2020 — Programming Languages — C++.

13. Contribución individual

Integrante	Contribución
Simon Ospina Ruiz	Elaboración y organización del informe, documentación de la práctica y supervisión general del código y del avance de la solución.
Samuel Lopez Velez	Desarrollo del Problema 2, correspondiente al búfer y limitador de tasa del firewall, incluyendo las estructuras y lógica asociadas.
Cristian Gustavo Buitrago Silva	Desarrollo del Problema 1, correspondiente al sistema de

	Undo/Redo y la lógica de las operaciones de edición y reversión.
--	--

Tabla 8. Contribución individual de los integrantes.

La distribución de tareas se hizo de acuerdo con la participación principal de cada integrante. Aun así, el trabajo se revisó entre todos para mantener una sola solución y una sola documentación.